

El encendido y apagado de los semáforos debe implementarse como un a máquina de estados Finitos (MEF) en la que cada luz verde debe habilitar el paso durante 30 ciclos de reloj, cada luz amarilla debe permanecer encendida durante 5 ciclos de reloj y cada luz roja debe permanecer encendida cuando las otras luces del mismo semáforo no están encendidas.

Diseño de Máquina de Estado

Para abordar el diseño de la solución a este problema utilizando máquina de Moore, comenzamos planteando los distintos estados de nuestra máquina:

Estado 0: R1, R2, A3

Estado 1: V1, R2, R3

Estado 2: A1, R2, R3

Estado 3: R1, V2, R3

Estado 4: R1, A2, V3

Estado 5: R1, R2, V3

Estado 6 (Inactivo): R1, R2, A3

Estado 7(Inactivo): R1, R2, A3

Donde 1, 2 y 3 representan a los semáforos. V representa que el semáforo se encuentra en verde, R representa que el semaforo se encuentra en rojo y A representa que el semaforo se encuentra en amarillo.

Cabe aclarar, que para el diseño de nuestros semaforos, la transicion de luces es: verde → amarillo → rojo → verde. Es decir, que la luz amarilla solo se enciende entre la luz verde y la roja, y no entre la roja y la verde.

En cuanto a las salidas asociadas a cada estado, generamos una salida binaria codificada de 9 bits. De esta forma, se representan 3 bits por semáforo, donde cada bit representa una luz (rojo, amarillo, verde).

Para cada semáforo, el bit más significativo representa la luz roja y el menos significativo la luz verde. El bit del medio la luz amarilla.

Se proporciona una tabla para visualizar la salida en cada estado:

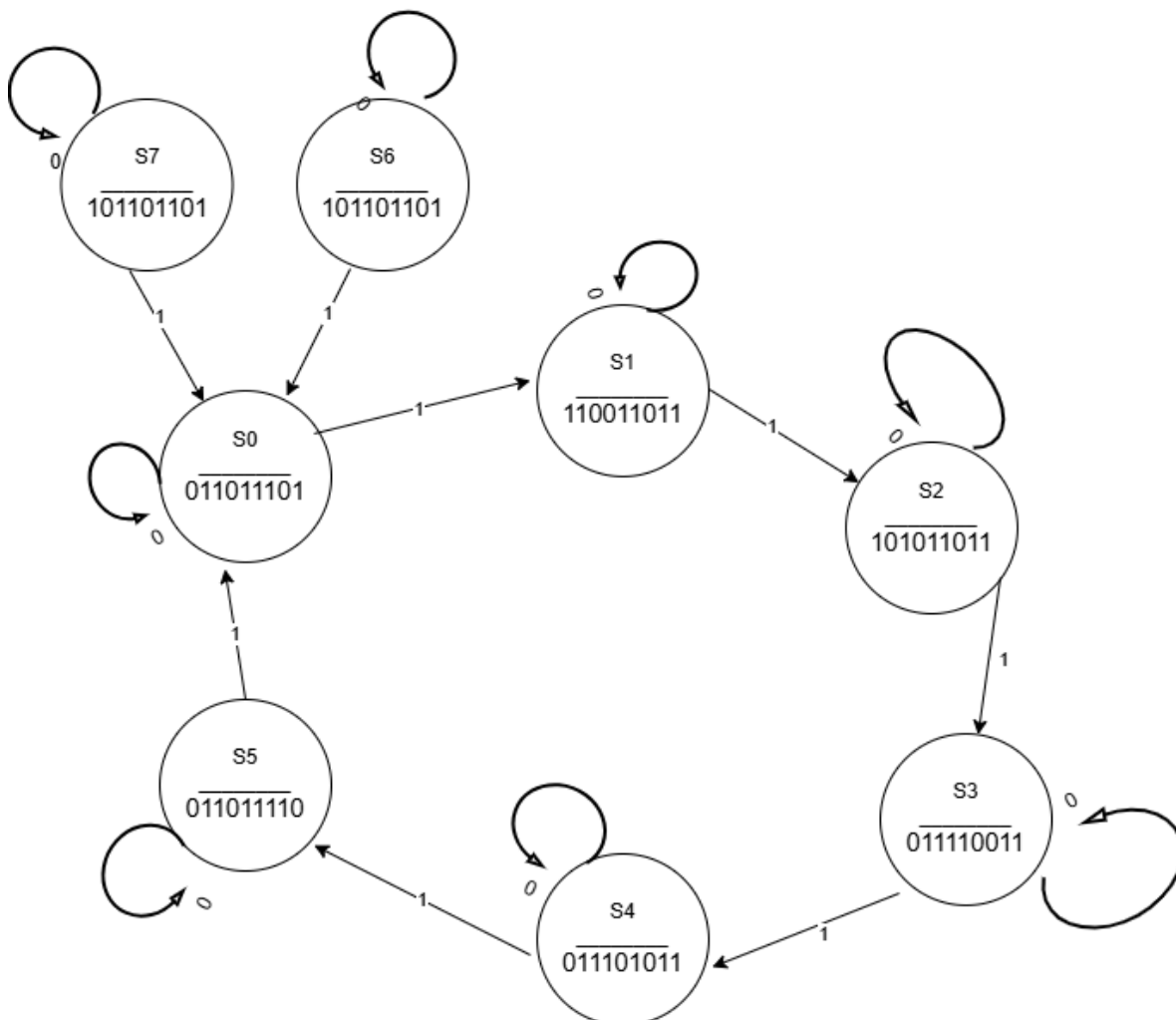
CONTADOR	STATE	STATE INFO	S1 (R,A,V)	S2 (R,A,V)	S3 (R,A,V)	OUT
0 < C <= 5	S0	R1, R2, A3	011	011	101	011011101
5 < C <=30	S1	V1, R2, R3	110	011	011	110011011
0 < C <= 5	S2	A1, R2, R3	101	011	011	101011011
5 < C <=30	S3	R1, V2, R3	011	110	011	011110011

$0 < C \leq 5$	S4	R1, A2, V3	011	101	011	011101011
$5 < C \leq 30$	S5	R1, R2, V3	011	011	110	011011110
Inactivo	S6	R1, R2, A3	011	011	101	011011101
Inactivo	S7	R1, R2, A3	011	011	101	011011101

Para controlar las transiciones de estado utilizamos un reloj que cuenta hasta 5 ciclos, luego hasta 30 y posteriormente se reinicia. Tanto a los 5 ciclos como a los 30 ciclos, envía una señal que cambia el estado de la máquina. Los 5 ciclos se utilizan para la luz amarilla y los 30 ciclos para la luz verde.

Para implementar nuestra máquina de estados, son necesarios 3 Flip Flops, ya que 2^3 es 8, entonces con 3 Flip Flops nos alcanza para poder representar a los 8 estados. Hay que tener en cuenta que existen 2 estados inactivos, los cuales implementaremos con una salida segura, donde se vuelve al estado inicial S0, reiniciando así el ciclo de la máquina.

Diagrama de Estados para Máquina de tipo Moore:



Lógica Combinatoria

Tabla de Transición de Estados

Aclaración: los estados se mantendrán hasta que se genere un pulso indicando el fin de un periodo de tiempo (5 ciclos o 30 ciclos dependiendo el estado que se encuentre la máquina), en el momento que llegue el pulso se transicionara al estado siguiente mediante las funciones lógicas mencionadas debajo.

Q2	Q1	Q0	Estado actual	Q2+	Q1+	Q0+	Estado siguiente
0	0	0	S0	0	0	1	S1
0	0	1	S1	0	1	0	S2
0	1	0	S2	0	1	1	S3
0	1	1	S3	1	0	0	S4
1	0	0	S4	1	0	1	S5
1	0	1	S5	0	0	0	S0
1	1	0	S6	0	0	0	S0
1	1	1	S7	0	0	0	S0
3 ff de logica ($2^n = \text{estados}$)				3 kmaps			

Aplicamos Mapa de Karnaugh para cada Flip Flop:

Para Flip Flop 2:

Q2Q1 \ Q0	00	01	11	10
0	0	0	0	1
1	0	1	0	0

Resultado

$$F = Q_0 \overline{Q_2} Q_1 + \overline{Q_0} Q_2 \overline{Q_1}$$

$$D_2 = Q_2' Q_1 Q_0 + Q_2 Q_1' Q_0$$

Para Flip Flop 1:

Q2Q1 \ Q0	00	01	11	10
0	0	1	0	0
1	1	0	0	0

Resultado

$$F = Q_0 \overline{Q_2} \overline{Q_1} + \overline{Q_0} \overline{Q_2} Q_1$$

$$D1 = Q2'(Q1 \text{ XOR } Q0)$$

Para Flip Flop 0:

Q2Q1 Q0	00	01	11	10
0	1	1	0	1
1	0	0	0	0

Resultado

$$F = \overline{Q0} \overline{Q1} + \overline{Q0} \overline{Q2}$$

$$D0 = Q0'(Q2' + Q1')$$

Los bits de salida (MSB: B8 ... LSB: B0) corresponden a cada uno de los leds de los tres semáforos. En la siguiente tabla se muestran los valores que se les asignará dependiendo el estado correspondiente.

Q2	Q1	Q0	Estado actual	B8(RS1)	B7(AS1)	B6(VS1)	B5(RS2)	B4(AS2)	B3(VS2)	B2(RS3)	B1(AS3)	B0(VS3)
0	0	0	S0	0	1	1	0	1	1	1	0	1
0	0	1	S1	1	1	0	0	1	1	0	1	1
0	1	0	S2	1	0	1	0	1	1	0	1	1
0	1	1	S3	0	1	1	1	1	0	0	1	1
1	0	0	S4	0	1	1	1	0	1	0	1	1
1	0	1	S5	0	1	1	0	1	1	1	1	0
1	1	0	S6	0	1	1	0	1	1	1	0	1
1	1	1	S7	0	1	1	0	1	1	1	0	1
3 ff de logica (2^n = estados)				bits de salida (9 kmaps)								

- Bit 8 → Luz Roja Semáforo 1
- Bit 7 → Luz Amarilla Semáforo 1
- Bit 6 → Luz Verde Semáforo 1
- Bit 5 → Luz Roja Semáforo 2
- Bit 4 → Luz Amarilla Semáforo 2
- Bit 3 → Luz Verde Semáforo 2
- Bit 2 → Luz Roja Semáforo 3
- Bit 1 → Luz Amarilla Semáforo 3
- Bit 0 → Luz Verde Semáforo 3

Aplicamos Mapa de Karnaugh para obtener la función lógica de cada bit de salida:

Para B8:

$\begin{matrix} Q_2 Q_1 \\ Q_0 \end{matrix}$	00	01	11	10
0	0	1	0	0
1	1	0	0	0

Resultado

$$F = Q_0 \overline{Q_2} \overline{Q_1} + \overline{Q_0} \overline{Q_2} Q_1$$

Simplificando: $B_8 = Q_2'(Q_1 \text{ XOR } Q_0)$

Para B7:

$\begin{matrix} Q_2 Q_1 \\ Q_0 \end{matrix}$	00	01	11	10
0	1	0	1	1
1	1	1	1	1

Resultado

$$F = Q_0 + Q_2 + \overline{Q_1}$$

$B_7 = Q_1' + Q_0 + Q_2$

Para B6:

$\begin{matrix} Q_2 Q_1 \\ Q_0 \end{matrix}$	00	01	11	10
0	1	1	1	1
1	0	1	1	1

Resultado

$$F = Q_2 + Q_1 + \overline{Q_0}$$

$$B6 = Q0' + Q1 + Q2$$

Para B5:

$\begin{matrix} Q2 & Q1 \\ \swarrow & \searrow \\ Q0 \end{matrix}$	00	01	11	10
0	0	0	0	1
1	0	1	0	0

Resultado

$$F = Q0 \overline{Q2} Q1 + \overline{Q0} Q2 \overline{Q1}$$

$$B5 = Q2'Q1Q0 + Q2Q1'Q0'$$

Para B4:

$\begin{matrix} Q2 & Q1 \\ \swarrow & \searrow \\ Q0 \end{matrix}$	00	01	11	10
0	1	1	1	0
1	1	1	1	1

Resultado

$$F = Q0 + Q1 + \overline{Q2}$$

$$B4 = Q2' + Q1 + Q0$$

Para B3:

$\begin{matrix} Q2 & Q1 \\ \swarrow & \searrow \\ Q0 \end{matrix}$	00	01	11	10
0	1	1	1	1
1	1	0	1	1

Resultado

$$F = Q2 + \overline{Q1} + \overline{Q0}$$

$$B3 = Q1' + Q0' + Q2$$

Para B2:

$\begin{matrix} Q2 & Q1 \\ \diagdown & / \\ Q0 \end{matrix}$	00	01	11	10
0	1	0	1	0
1	0	0	1	1

Resultado

$$F = Q0 \cdot Q2 + Q2 \cdot Q1 + \overline{Q0} \cdot \overline{Q2} \cdot \overline{Q1}$$

$$B2 = Q2(Q1 + Q0) + Q2'Q1'Q0'$$

Para B1:

$\begin{matrix} Q2 & Q1 \\ \diagdown & / \\ Q0 \end{matrix}$	00	01	11	10
0	0	1	0	1
1	1	1	0	1

Resultado

$$F = Q0 \cdot \overline{Q2} + Q2 \cdot \overline{Q1} + \overline{Q2} \cdot Q1$$

Simplificando: $B1 = Q2'Q0 + Q2 \text{ XOR } Q1$

Para B0:

$\begin{matrix} Q2 & Q1 \\ \diagdown & / \\ Q0 \end{matrix}$	00	01	11	10
0	1	1	1	1
1	1	1	1	0

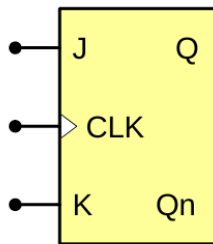
Resultado

$$F = Q1 + \overline{Q2} + \overline{Q0}$$

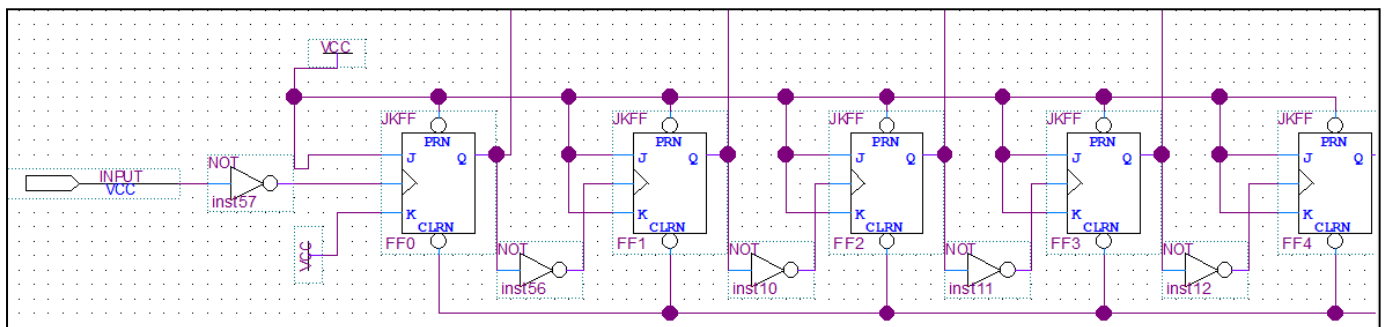
$$B0 = Q2' + Q1 + Q0'$$

Implementación de la Máquina de Estados

Comenzamos diseñando el contador de pulsos para poder llevar a cabo la transición de estados en nuestra máquina. El contador implementado está compuesto por 5 Flip Flops JK. Los FF JK pueden ser utilizados como contadores cuando sus dos entradas J y K están constantemente activas en 1:

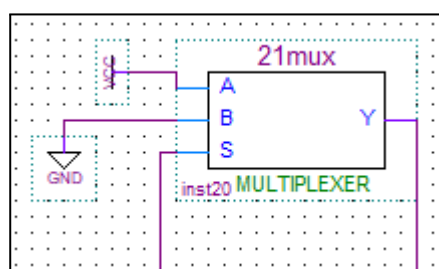


J	K	CLK	Q
0	0	↑	Q (Sin cambios)
0	1	↑	0
1	0	↑	1
1	1	↑	$\overline{Q}$ (Toggle)

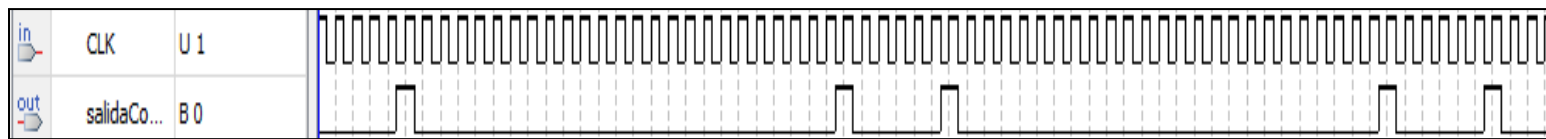


De esta forma, se colocan los FF en serie. El primer FF recibe en CLK una onda cuadrada con pulsos periódicos, y luego, cada FF siguiente recibe en CLK, la salida Q negada del FF anterior. Cabe aclarar que utilizamos Flip Flops activados con flancos de bajada, por lo que nuestro contador resulta ascendente.

Una vez creado nuestro contador, implementamos lógica para que nos devuelva un pulso cuando cuenta 5 ciclos, y otro pulso cuando cuenta 30 ciclos, y posteriormente, que se reinicie al llegar a este valor. Para reiniciar el contador, se utilizó un multiplexor, que cuando recibe la señal de reinicio (en S), envía una señal en bajo a todas las entradas CLR de los Flip Flops.



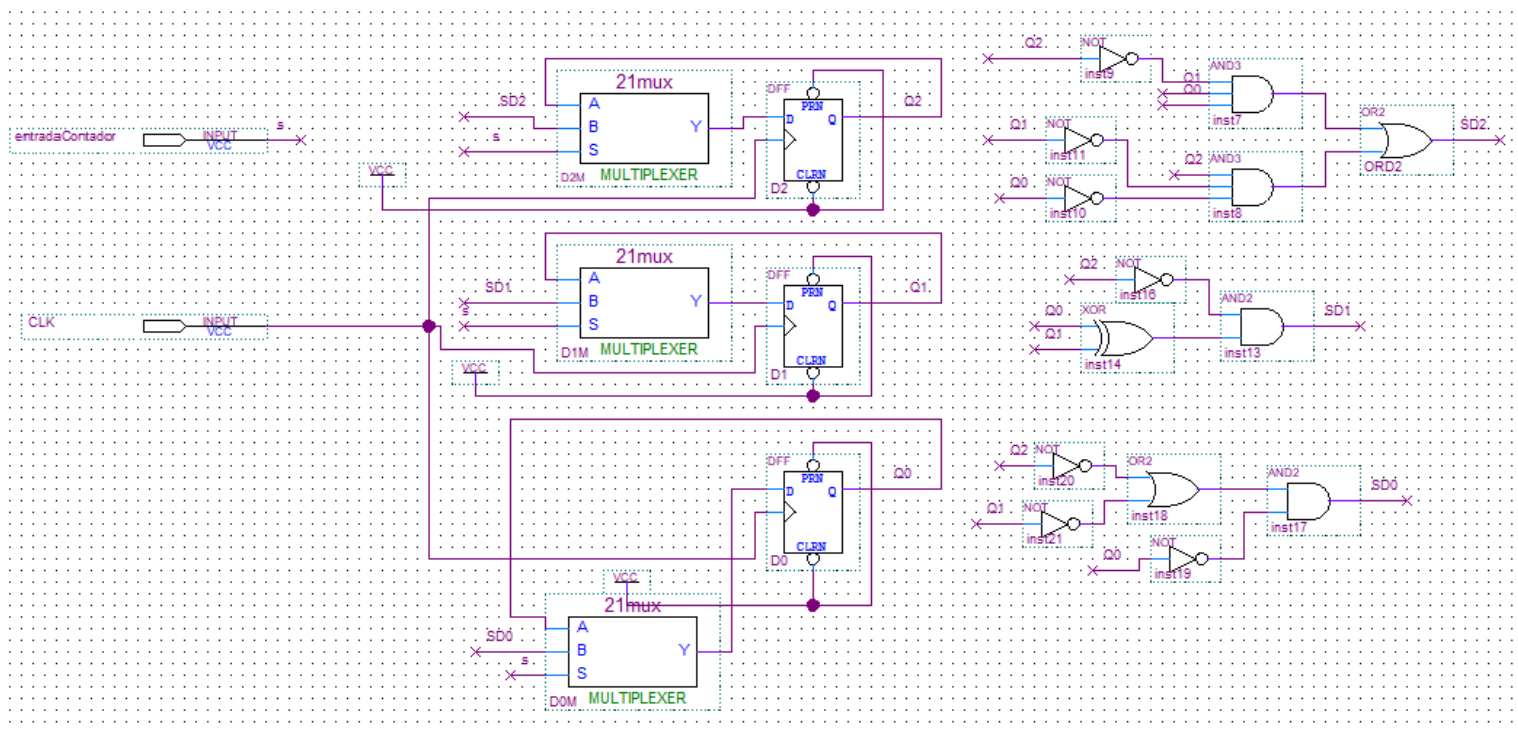
A continuación, se muestra el funcionamiento del contador en la interfaz gráfica de una WaveForm en Quartus:



Se observa que la entrada al contador CLK es una señal cuadrada periódica. Como se mencionó anteriormente, los pulsos en la salida se generan al contar 5 y 30 ciclos de la señal de entrada, luego el contador se reinicia y repite.

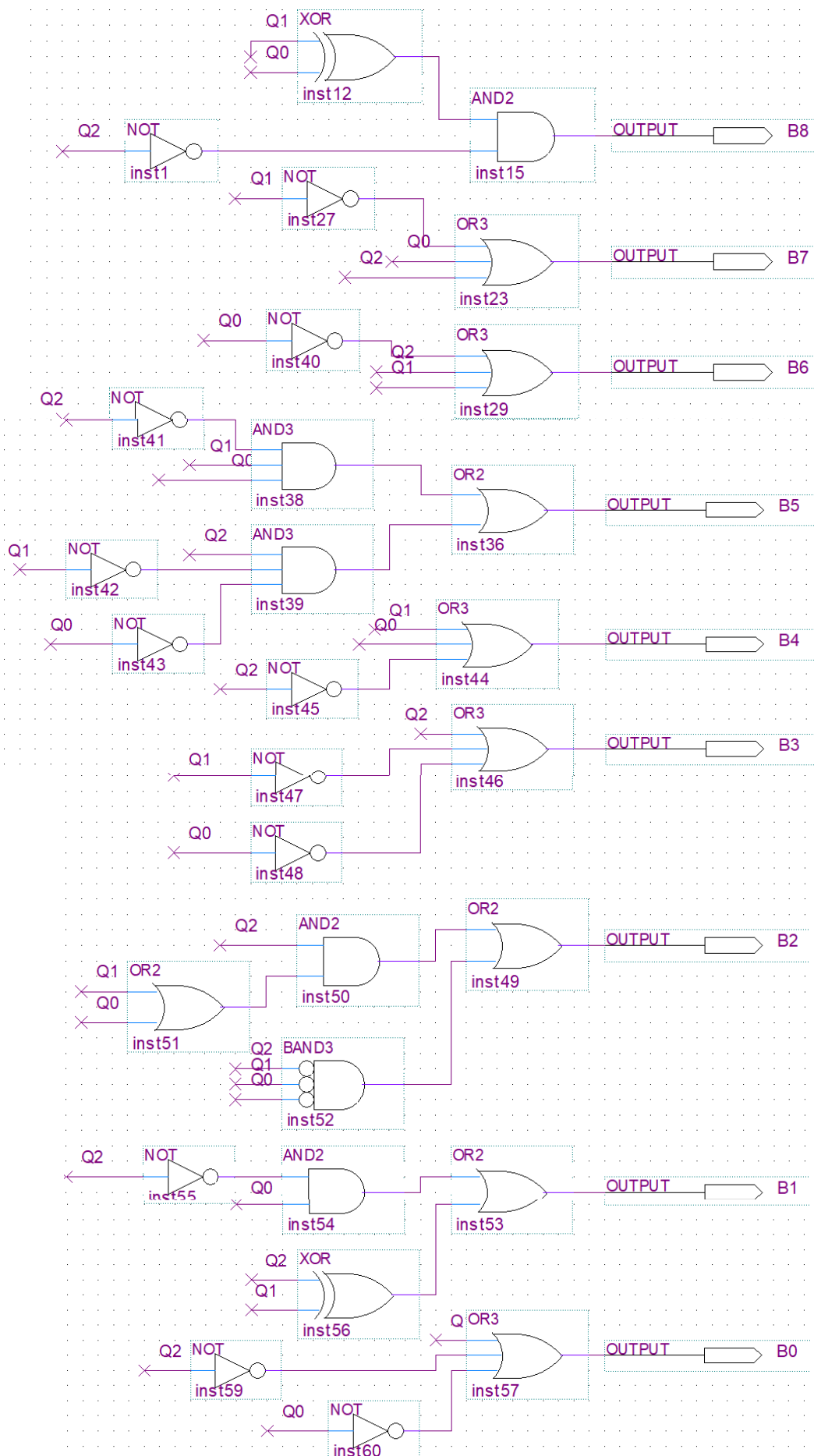
Después de implementar el contador, se generó en un diagrama de bloques aparte la máquina de estados. Para diseñar la máquina de estados nos basamos en las funciones lógicas obtenidas anteriormente, donde las salidas SD1, SD2, SD3 representan los siguientes bits de estado para cada flip flop.

El multiplexor actúa de la misma manera que comentamos anteriormente, activando el cambio de estado (B) al momento que le llega una señal desde el contador, y donde mientras no llegue dicha señal, mantenga los bits que tenía en el estado anterior. Con este componente, nos aseguramos que el cambio de estado se produzca sólo cuando se active un pulso del contador y no en cada flanco de reloj.

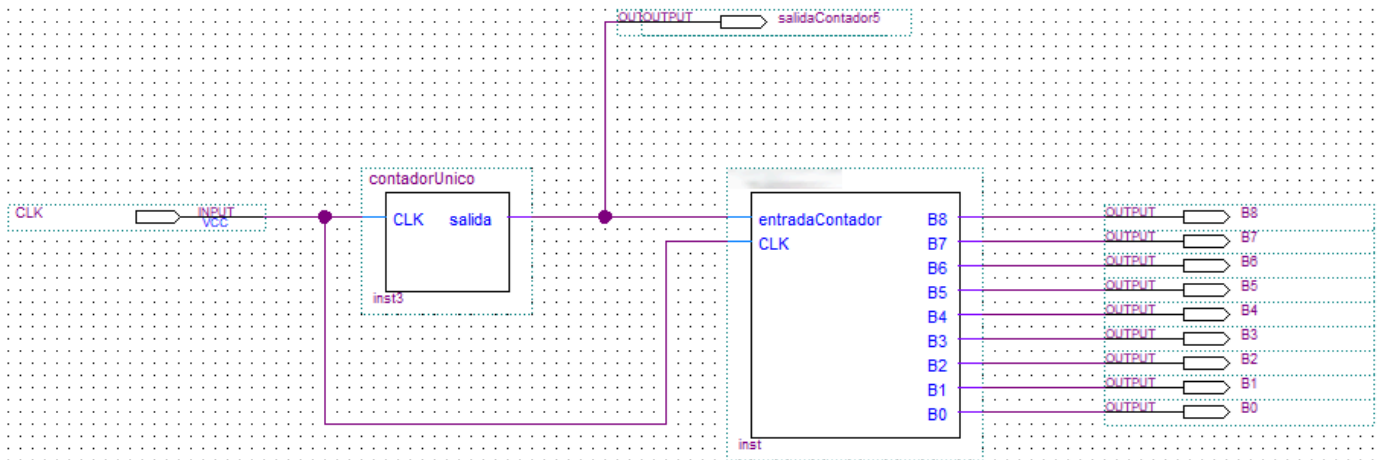


Las entradas son: la señal de salida del contador, y una señal cuadrada periódica que se utiliza en los Flip Flops para su correcto funcionamiento.

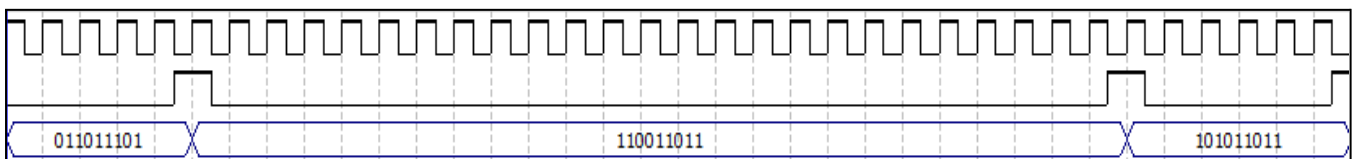
Se implementa la lógica de cada bit de salida de acuerdo a lo obtenido anteriormente:



Por último, generamos un componente del contador y otro de la máquina, para probarlos en conjunto en un nuevo diagrama de bloques:



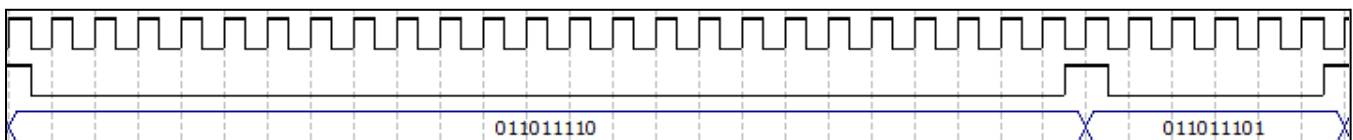
A continuación, se muestra el funcionamiento de las transiciones de estado de la máquina en conjunto con el contador de ciclos, en la interfaz gráfica de una Waveform en Quartus:



- En los primeros 5 ciclos, la salida es 011011101 (estado 0).
- Después, por 30 ciclos, la salida es 110011011 (estado 1).
- Los siguientes 5 ciclos, la salida es 101011011 (estado 2).



- Los siguientes 30 ciclos, la salida es 011110011 (estado 3).
- Los siguientes 5 ciclos, la salida es 011101011 (estado 4).



- Los siguientes 30 ciclos, la salida es 011011110 (estado 5).
- Luego se repite el ciclo, donde en los siguientes 5 ciclos, la salida es 011011101 (estado 0).